Directrices mínimas para la elección del tema del Proyecto Semestral

Asignatura: Desarrollo Fullstack I

Las siguientes directrices establecen los **requisitos mínimos que debe cumplir un proyecto semestral para ser aceptado** dentro de la asignatura.
Estos lineamientos buscan asegurar un **nivel de complejidad acorde al tercer semestre de la carrera**, garantizando que todos los proyectos desarrollados en las distintas sedes mantengan un estándar académico equivalente.

Es importante considerar que **los requisitos aquí indicados representan el mínimo exigido**, por lo que los equipos pueden desarrollar proyectos con mayor

Contexto del proyecto

El tema del proyecto debe cumplir las siguientes condiciones:

- Representar **un problema real o una necesidad concreta de un sistema de información**.
- Corresponder a **un sistema completo de gestión o servicio digital**, evitando propuestas excesivamente simples o meramente académicas.
- Tener **un nivel de complejidad acorde a estudiantes de tercer semestre**, quienes ya dominan los fundamentos de programación y bases de datos.
- Incluir **múltiples módulos funcionales y componentes backend** que justifiquen el uso de arquitectura de microservicios.

Arquitectura del sistema

El sistema deberá desarrollarse **obligatoriamente utilizando arquitectura de microservicios con Spring Boot**.

El proyecto debe considerar como mínimo:

- **10 microservicios independientes**.
- Cada microservicio debe:
    - Tener **una responsabilidad clara dentro del sistema**.
    - Manejar **su propia lógica de negocio**.
    - Exponer **API REST para su consumo**.
    - Contar con **sus propios endpoints funcionales**.
- Los microservicios deben estar **completamente desacoplados entre sí**.
- La cantidad indicada corresponde al **mínimo exigido**, pudiendo implementarse más microservicios si el proyecto lo requiere.

Comunicación entre microservicios

El sistema debe demostrar **interacción real entre los distintos microservicios**, incluyendo:

- Consulta de datos entre servicios.
- Validación de información entre microservicios.
- Flujo de datos entre distintos módulos del sistema.
- Interacción mediante mecanismos de comunicación apropiados, tales como:
    - WebClient
    - u otros mecanismos equivalentes de comunicación entre servicios.

El sistema debe contar además con un **API Gateway**, el cual puede implementarse utilizando herramientas como **Eureka u otros mecanismos equivalentes de gestión de servicios**.

Persistencia de datos

Cada microservicio debe contar con **su propia base de datos independiente**.

Se deben cumplir las siguientes condiciones:

- No se permite **compartir tablas entre microservicios**.
- No se deben duplicar estructuras de datos entre servicios.
- Las bases de datos deben estar **correctamente normalizadas**.
- Debe existir un **modelo relacional validado mediante diagramas de base de datos**.

Los motores de base de datos permitidos son:

- **MySQL (recomendado)**
- **Oracle**

Los entornos sugeridos para su implementación incluyen:

- XAMPP
- Laragon
- u otros entornos equivalentes que permitan gestionar bases de datos locales.

El proyecto debe evidenciar:

- entidades correctamente modeladas
- relaciones entre entidades
- integridad de los datos

- validaciones y reglas de negocio aplicadas a nivel de aplicación.

Roles de usuario

El sistema debe contemplar **entre 2 y 3 roles de usuario diferenciados**.

Cada rol debe:

- tener **funcionalidades distintas dentro del sistema**
- manejar **permisos y privilegios diferentes**
- interactuar con **distintos módulos del sistema**

Ejemplos de roles:

- Administrador
- Usuario cliente
- Operador del sistema

Funcionalidades del sistema

El sistema debe incluir funcionalidades suficientes para justificar su arquitectura.

Como mínimo debe considerar:

- múltiples operaciones **CRUD**
- operaciones **personalizadas adicionales a los CRUD básicos**
- **validaciones de datos**
- **reglas de negocio**
- **consultas a bases de datos**
- **flujo de información entre microservicios**

Las APIs desarrolladas deben permitir **gestionar completamente los procesos del sistema**.

Seguridad básica del sistema

El sistema debe implementar **mecanismos básicos de seguridad**, tales como:

- encriptación de contraseñas antes de almacenarlas
- autenticación de usuarios
- generación de tokens de sesión en el proceso de login
- validación de accesos según roles de usuario.
- Cualquier otro que garantice la seguridad, protección y acceso de datos e información.

Control de versiones y repositorio

Todos los proyectos deben utilizar obligatoriamente:

GitHub como repositorio de código.

El repositorio debe permitir evidenciar:

- avance progresivo del desarrollo
- participación de todos los integrantes del equipo
- historial de cambios del proyecto.

Se recomienda trabajar con:

- ramas de desarrollo
- commits frecuentes
- estructura organizada del proyecto.

Documentación, pruebas y despliegue (Experiencia de Aprendizaje 3)

Durante el desarrollo del proyecto se deberán incorporar elementos adicionales tales como:

- documentación técnica del sistema
- pruebas unitarias
- despliegue de los microservicios.

El despliegue puede realizarse mediante:

- entorno local
- herramientas de exposición de puertos
- servicios gratuitos de despliegue.

El sistema debe quedar disponible mediante **una URL que permita consumir las APIs del proyecto**.

jemplos de proyectos aceptables

Los siguientes ejemplos ilustran el tipo de sistemas que cumplen con el nivel de complejidad esperado:

- Sistema de gestión de reservas de hoteles
- Plataforma de gestión de pedidos para restaurantes
- Sistema de gestión de bibliotecas
- Plataforma de gestión de ventas online
- Sistema de gestión de inventario con logística

- Marketplace de servicios o productos
- Plataforma de gestión de reservas de servicios

Ejemplos de proyectos no aceptables

No se aceptarán proyectos excesivamente simples, tales como:

- listas básicas de tareas
- agendas simples
- CRUD simples sin lógica de negocio
- sistemas que no requieran interacción entre microservicios
- aplicaciones con una única base de datos compartida.